Artificial and Computational Intelligence

AIMLCZG557

BITS Pilani

Pilani|Dubai|Goa|Hyderabad

ACI Team

innovate achieve lead

BITS Pilani

Pilani|Dubai|Goa|Hyderabad

AIMLCZG557 – CS#5
Local Search Algorithms and Introduction to Evolutionary Algorithms

Agenda for CS #5

1) Recap of CS #4
2) Local Search Algorithms
3) Hill Climbing
4) Local Beam Search
5) Online Search Agents
6) Genetic Algorithms
7) Introduction to ACO
8) Q&A!

Local Search & Optimization

Optimization is central to Artificial Intelligence. Many AI problems require finding the best solution among a vast number of possibilities!

Path vs State Optimization

- Previous topics: path to goal is solution to problem
  - systematic exploration of search space.
- This topic (local search): a *state* is solution to the problem!
  - for some problems path is irrelevant.
  - E.g., 8-queens
- In many optimization problems, the path to the goal is irrelevant; the goal state itself is the solution
- State space = set of "complete" configurations
  - Find configuration satisfying constraints, e.g., n-queens
- In such cases, we can use local search algorithms
  - Keep a single "current" state, try to improve it

Local search and optimization

- Local search
  - Keep track of single current state
  - Move only to neighboring states
  - Ignore paths
- Advantages:
  - Use very little memory
  - Can often find reasonable solutions in large or infinite (continuous) state spaces.
- "Pure optimization" problems
  - All states have an objective function
  - Goal is to find state with max (or min) objective value
  - Does not quite fit into path-cost/goal-state formulation
  - Local search can do quite well on these problems.

Optimization Models

- Local search algorithms – Search in the state-space in the neighborhood of current position until an optimal solution is found
  - Hill Climbing Algorithm and its variants
  - Simulated Annealing
  - Local Beam Search
- Population based algorithms – Instead of single instance in state-space, use multiple parallel instances (i.e., population) in finding the optimal solution
  - Genetic Algorithms, Ant colony optimization

Local Search

- Local search algorithms focus on improving a single current state rather than constructing complete search trees.
- They are especially useful when only the final solution matters, not the path to the solution.
- Applications:
  - Hyperparameter tuning in machine learning
  - Feature selection
  - Robotics path planning
  - Scheduling and timetabling
  - VLSI design

Optimization Models

innovate achieve lead

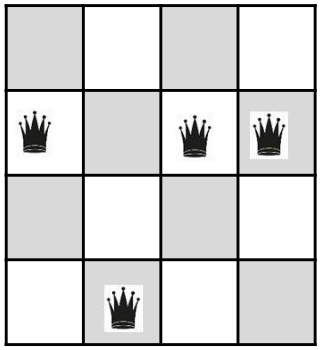
- **Goal** : Navigate through a state space for a given problem such that an optimal solution can be found
- **Objective** : Minimize or Maximize the objective evaluation function value
- **Scope** : Local
- **Objective Function** : Fitness Value evaluates the goodness of current solution
- **Local Search** : Search in the state-space in the neighbourhood of current position until an optimal solution is found

Single Instance Based	Multiple Instance Based
Hill Climbing ✅	Genetic Algorithm ✅
Local Beam Search ✅	Ant Colony Optimization ✅
Simulated Annealing	Particle Swarm Optimization
Tabu Search	

Local Search Terminology

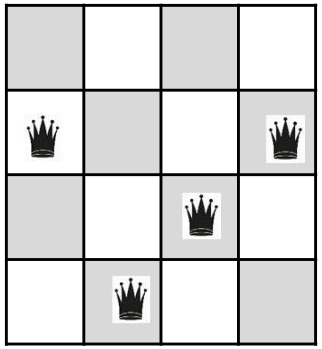
- **Local Search:** Search in the state-space in the neighborhood of current position until an optimal solution is found.

Feasible State/Solution

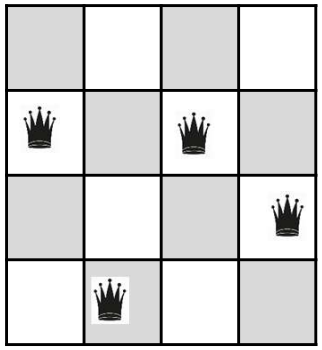


V1	V2	V3	V4
2	4	2	2

Neighbour States

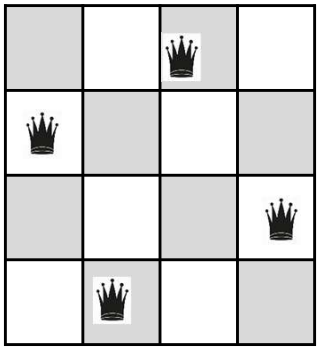


V1	V2	V3	V4
2	4	3	2



V1	V2	V3	V4
2	4	2	3

Optimal Solution

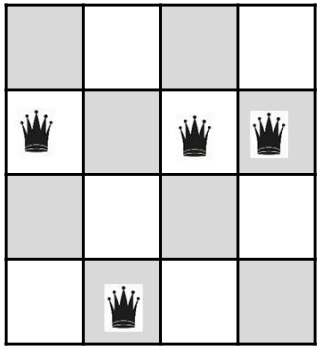


V1	V2	V3	V4
2	4	1	3

Fitness Value: h(n)= Number of conflicting pairs of queens

h(n)= 4 h(n)= 4 h(n)= 2 h(n)= 0

Local Search Expansion



V1	V2	V3	V4
2	4	2	2

1NN:

(2-1) (2-3) (2-4)
(4-1) (4-2) (4-3)
(2-1) (2-3) (2-4)
(2-1) (2-3) (2-4)

2NN:

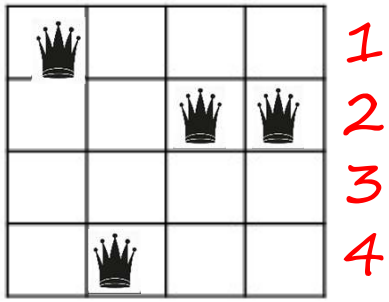
V1 V2 → 9
V1 V3 → 9
V1 V4 → 9
V2 V3 → 9
V2 V4 → 9
V3 V4 → 9

Hill Climbing

1. Select a random state
2. Evaluate the fitness scores for all the successors of the state

h(n) = Number of non-conflicting pairs of queens in the board.

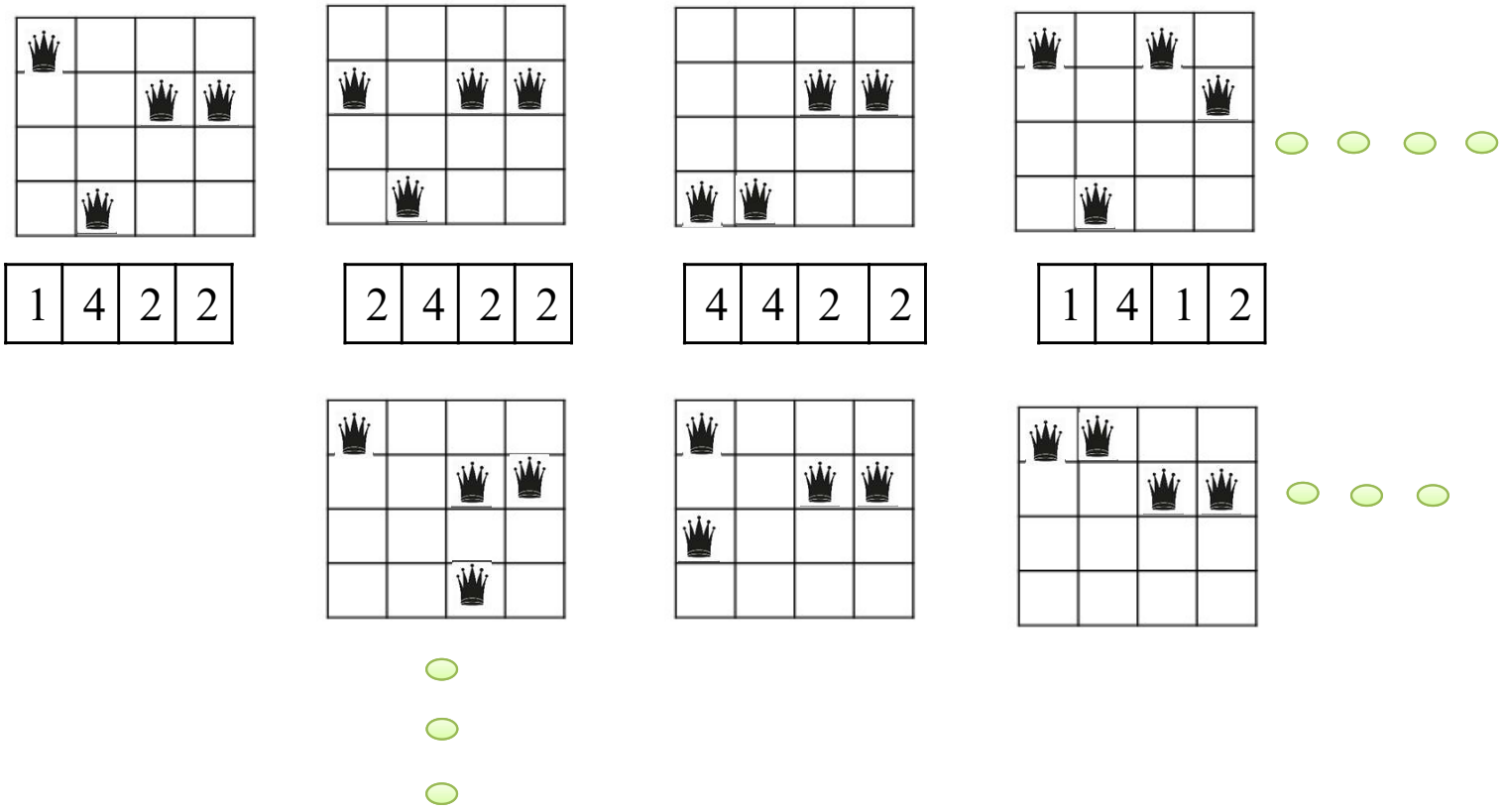
N Q1-Q2
N Q1-Q3
N Q1-Q4
N Q2-Q3
Y Q2-Q4
Y Q3-Q4



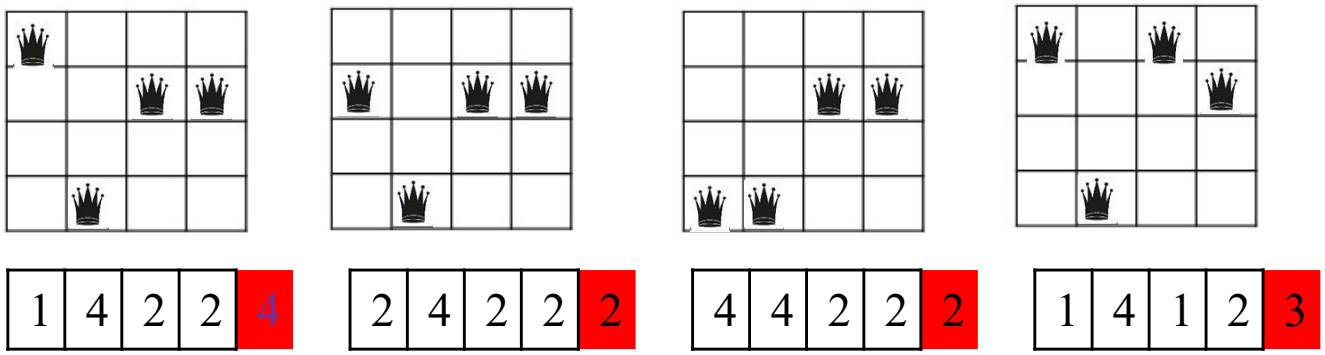
1	4	2	2	4

Hill Climbing

1. Select a random state
2. Evaluate the fitness scores for all the successors of the state



Hill Climbing

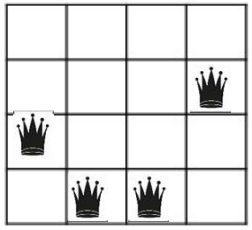


Hill Climbing

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
    current ← problem.INITIAL
    while true do
        neighbor ← a highest-valued successor state of current
        if VALUE(neighbor) ≤ VALUE(current) then return current
        current ← neighbor
```

Hill Climbing with Random Restart

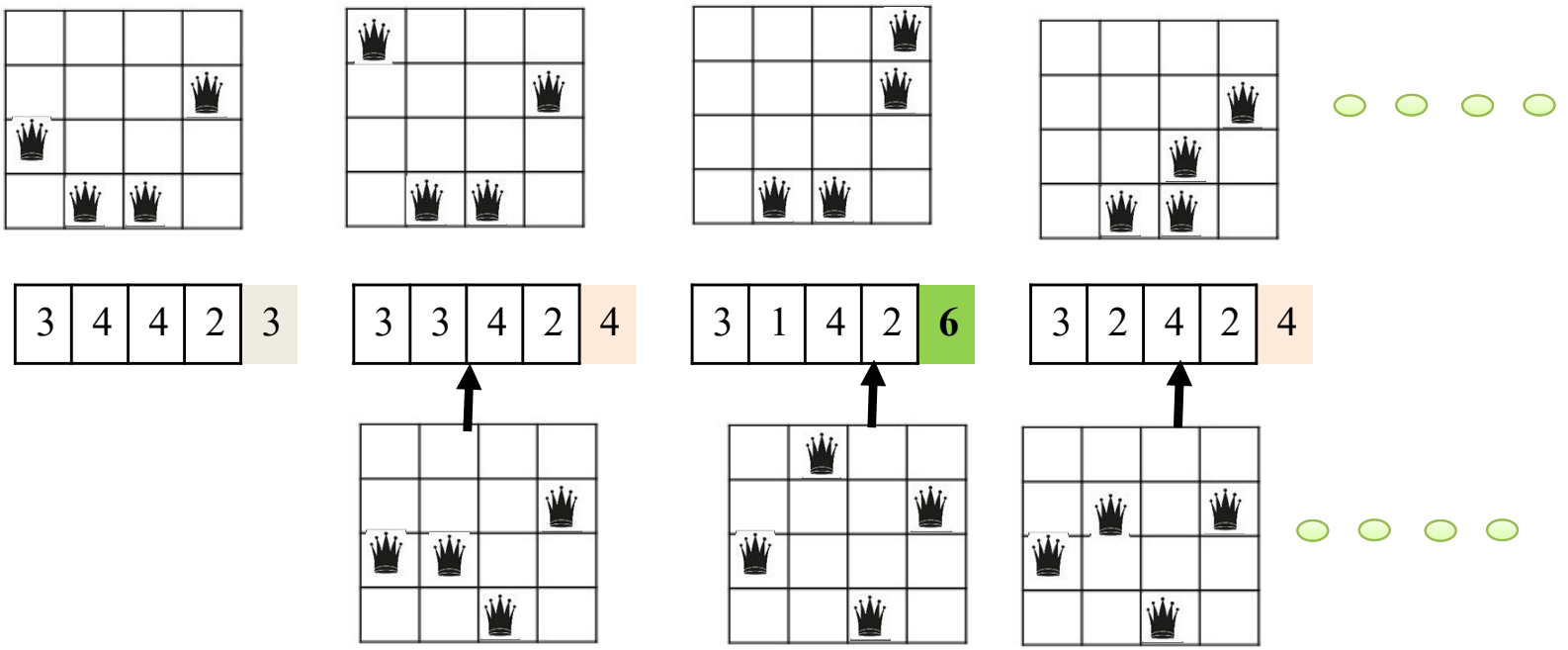
1. Select a random state
2. Evaluate the fitness scores for all the successors of the state
3. Repeat from Step 2



3	4	4	2	3

Hill Climbing with Random Restart

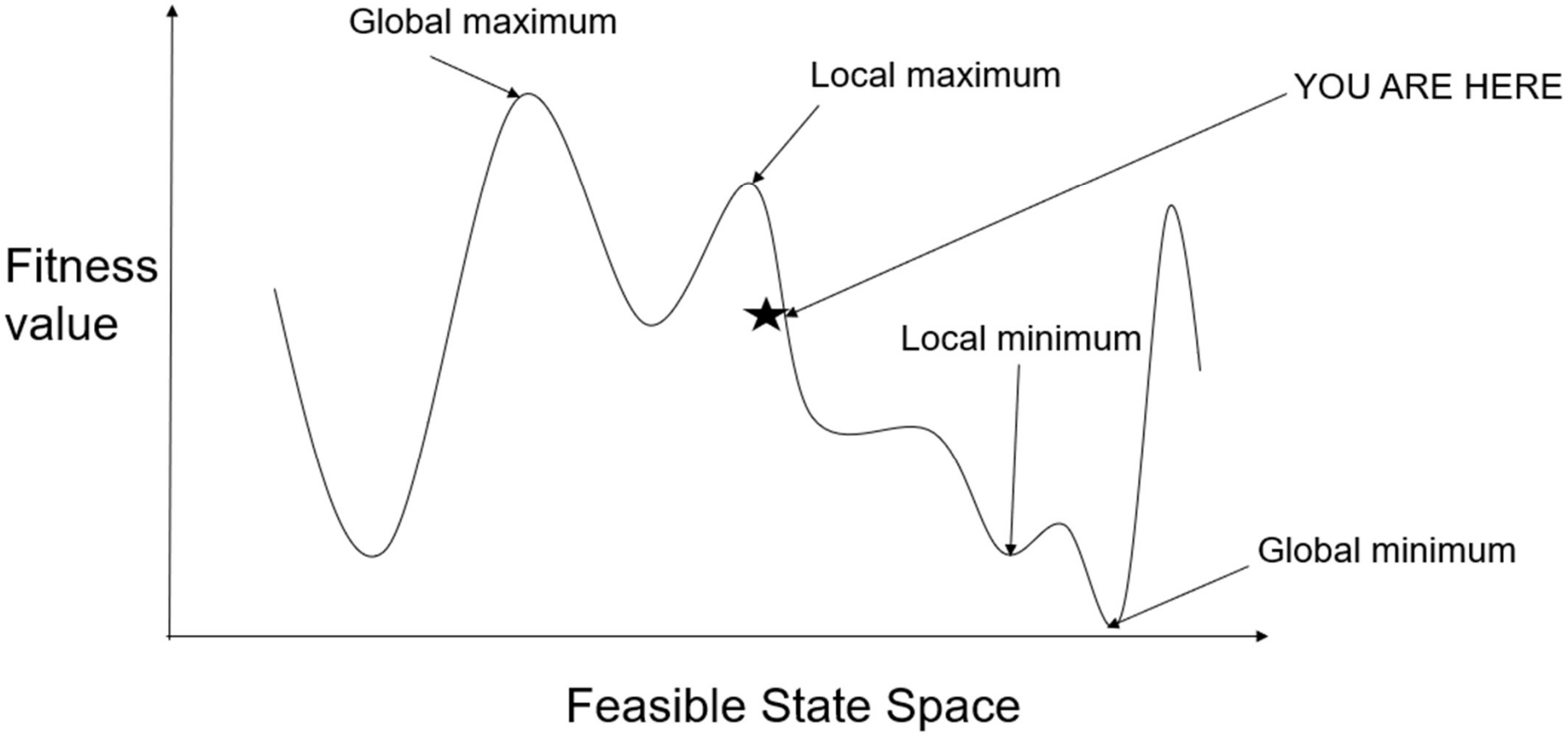
1. Select a random state
2. Evaluate the fitness scores for all the successors of the state
3. Repeat from Step 2



Hill Climbing with Random Restart (When to Stop?)

- Termination Condition 1: When no successor has a better fitness value
- Termination Condition 2: When we attained Global Maxima or Global minima
- Termination Condition 3: When a predefined value of ‘k’ denoting the number of restart is reached.

Hill Climbing

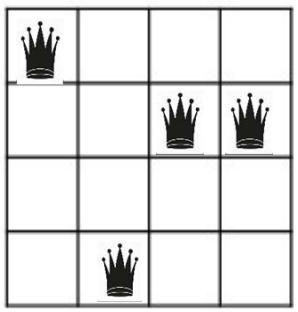


Stochastic Hill Climbing

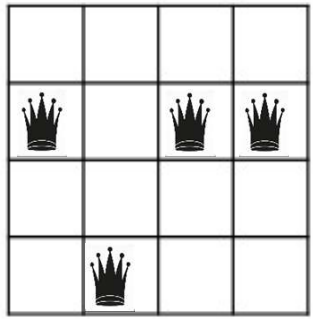
$next \leftarrow$ a randomly selected successor of $current$
$\Delta E \leftarrow next.\text{VALUE} - current.\text{VALUE}$
if $\Delta E > 0$ **then** $current \leftarrow next$
else $current \leftarrow next$ only with probability $e^{\Delta E/T}$

Stochastic Hill Climbing

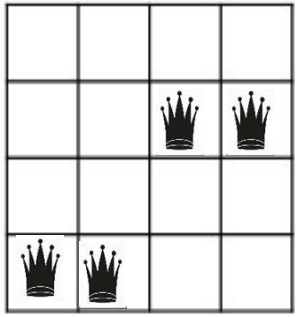
1. Select a random state
2. Evaluate the fitness scores for all the successors of the state
3. Calculate the *probability* of selecting a successor based on fitness score
4. Select the next state based on the highest probability
5. Repeat from Step 2



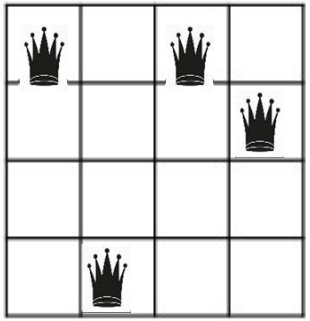
1	4	2	2	4
Probability=0.08				



2	4	2	2	2
Probability=0.42				



4	4	2	2	2
Probability=0.42				



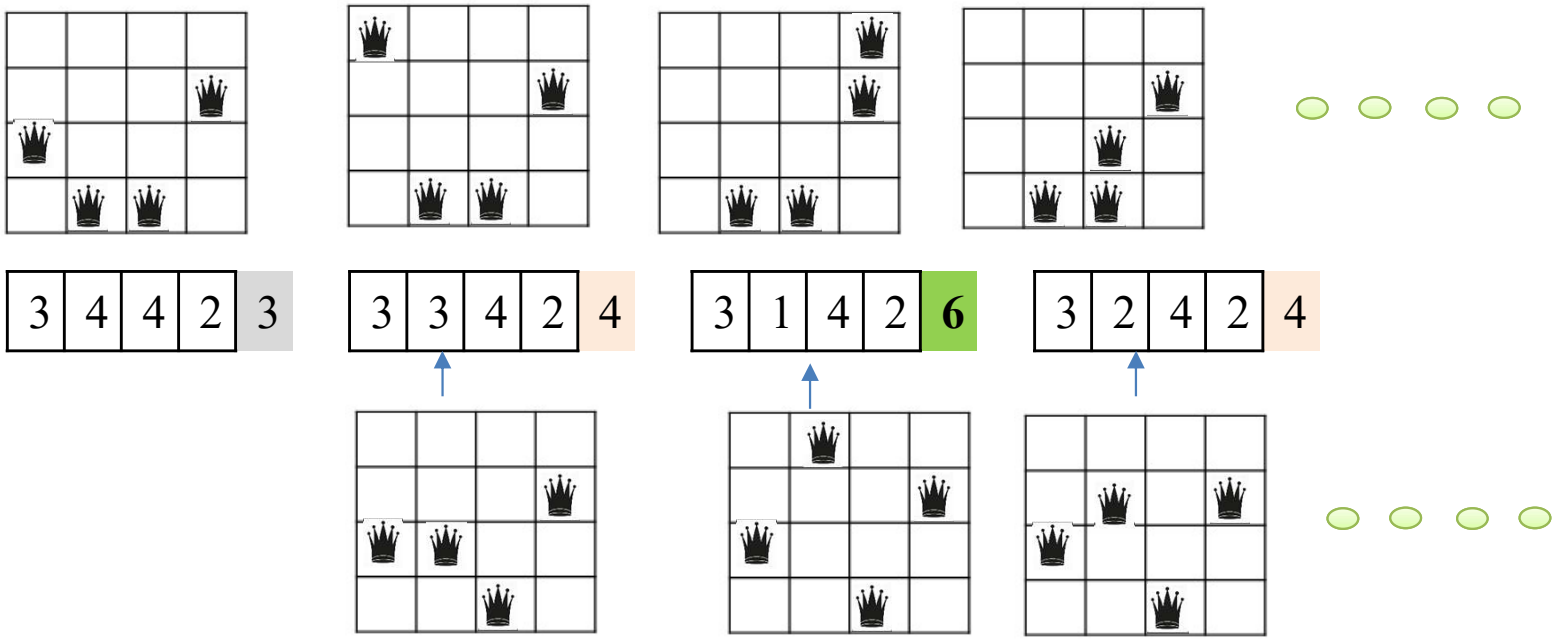
1	4	1	2	2
Probability=0.33				

12 N = {4,2,2,3,3,2,1,3,2,1,3,2}

F(n)	Freq	Prob
1	2	2/12
2	5	5/12
3	4	4/12
4	1	1/12

Beam Search

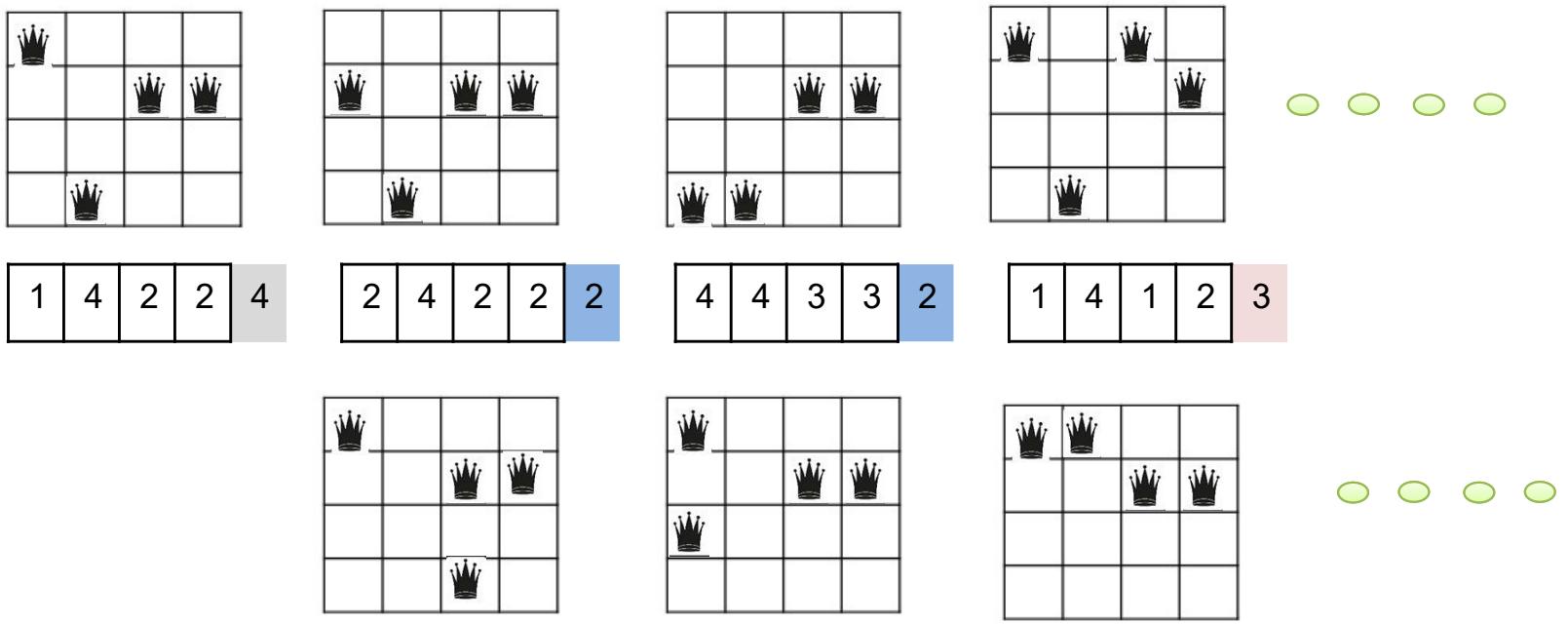
1. Initialize **k** random state
2. Evaluate the fitness scores for all the successors of the k states
3. Calculate the probability of selecting a successor based on fitness score
4. Select the next state based on the highest probability
5. If the goal is not found, Select the next ‘k’ states randomly based on the probability
6. Repeat from Step 2



Stochastic Beam Search

Sample from 1st State

1. Initialize k random state
2. Evaluate the fitness scores for all the successors of the k states
3. Calculate the probability of selecting a successor based on fitness score
4. Select the next state based on the highest probability
5. If the goal is not found, Select the next ‘k’ states randomly based on the probability
6. Repeat from Step 2



Genetic Algorithms

- A genetic algorithm (or GA) is a search technique used in computing to find true or approximate solutions to optimization and search problems.
- (GA)s are categorized as global search heuristics.
- (GA)s are a particular class of evolutionary algorithms that use techniques inspired by evolutionary biology such as inheritance, mutation, selection, and crossover (also called recombination).
- The evolution usually starts from a population of randomly generated individuals and happens in generations.
- In each generation, the fitness of every individual in the population is evaluated, multiple individuals are selected from the current population (based on their fitness), and modified to form a new population.

Genetic Algorithms

- The new population is used in the next iteration of the algorithm.
- The algorithm terminates when either a maximum number of generations has been produced, or a satisfactory fitness level has been reached for the population.
- No convergence rule or guarantee

Genetic Algorithms Vocabulary

- **Population** - Group of all individuals
- **Individual** - Any possible solution
- **Fitness** – Target function that we are optimizing (each individual has a fitness)
- **Trait** - Possible aspect (features) of an individual
- **Genome** - Collection of all chromosomes (traits) for an individual.

Genetic Algorithms

Population:

- Bio: Collection of Individuals.
- Algorithm: Collection of states.

Fitness:

- Bio: More healthy, less prone to diseases.
- Algorithm: Closest to the final solution.

Selection:

- Bio: Selecting species that are the most biological fit.
- Algorithm: Selecting states that are closest to the solution (fittest).

Crossover:

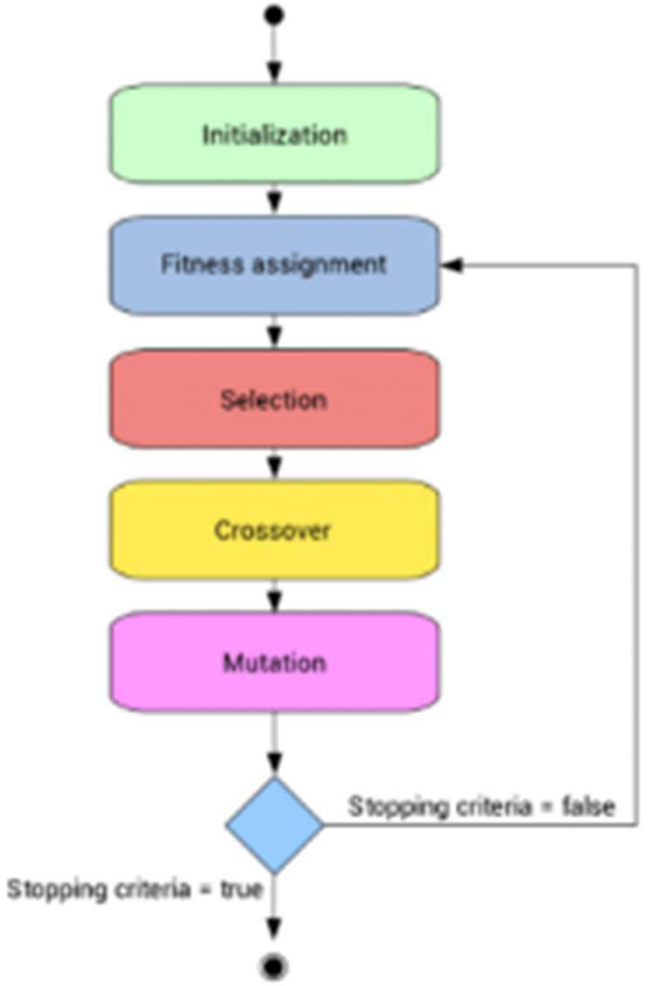
- Bio: Mating or reproducing
- Algorithm: Interchanging values between selected states

Mutation:

- Bio: Change or variation in DNA
- Algorithm: Alternation of state (randomly)

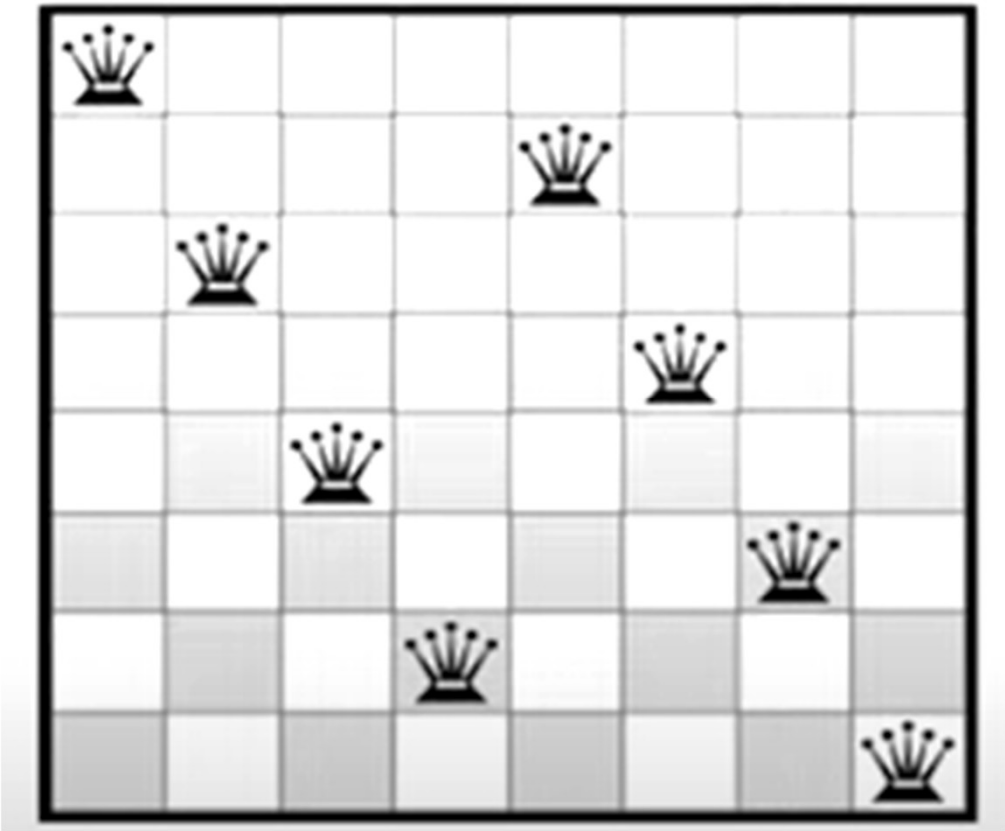
Basic Genetic Algorithm

- Start with a large "population" of randomly generated "attempted solutions" to a problem
- Repeatedly do the following:
  - Evaluate each of the attempted solutions
  - (probabilistically) keep a subset of the best solutions
  - Use these solutions to generate a new population
  - Quit when you have a satisfactory solution (or you run out of time)



8-Queens Problem

- Arrange 8 queens in such a way that no queen attacks each other.

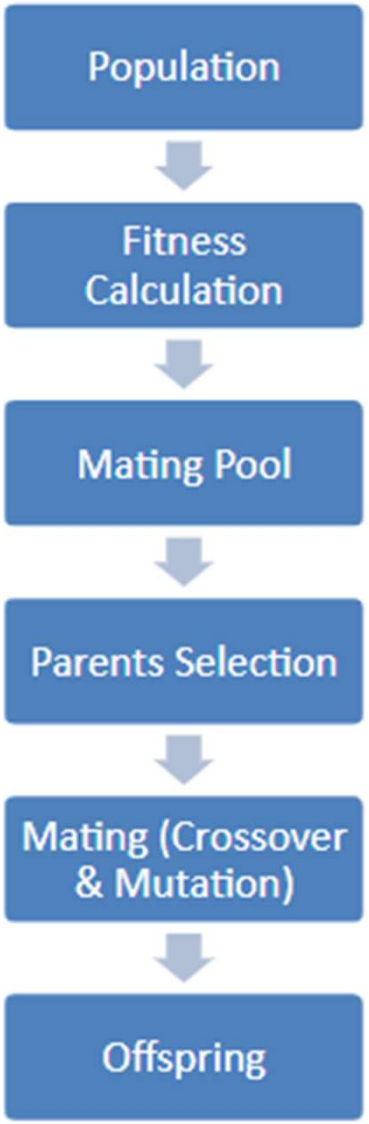


Solving 8-Queens Problem using Genetic Algorithm

- Step 01: Representing individuals
- Step 02: Generating an initial population.
- Step 03: Applying a fitness function.
- Step 04: Selecting parents for mating in accordance to their fitness.
- Step 05: Crossover of parents to produce new generation
- Step 06: Mutation of new generation to bring diversity
- Step 07: Repeat until solution is reached.

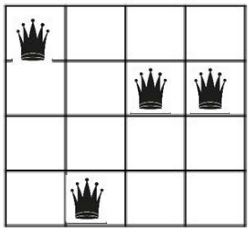
Genetic Algorithm

- Step 01: Representing individuals
- Step 02: Generating an initial population.
- Step 03: Applying a fitness function.
- Step 04: Selecting parents for mating in accordance to their fitness.
- Step 05: Crossover of parents to produce new generation
- Step 06: Mutation of new generation to bring diversity
- Step 07: Repeat until solution is reached.



Step 01: Representing Individuals

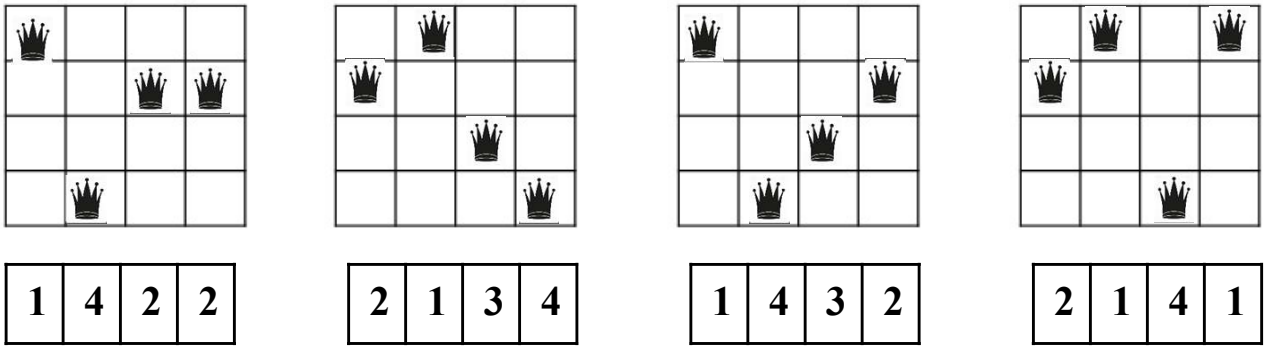
1. Select ‘k’ random states – Initialization : k=4
2. Evaluate the fitness value all states
3. If anyone of the state’s has achieved the threshold fitness value or threshold new states or no change is seen than previous iteration then the algorithm stops
4. Else, use roulette wheel mechanism to select pair/s
5. Pairs selected produces new state (successor) by crossover
6. Successor is allowed to mutate
7. Repeat from Step 2



1	4	2	2

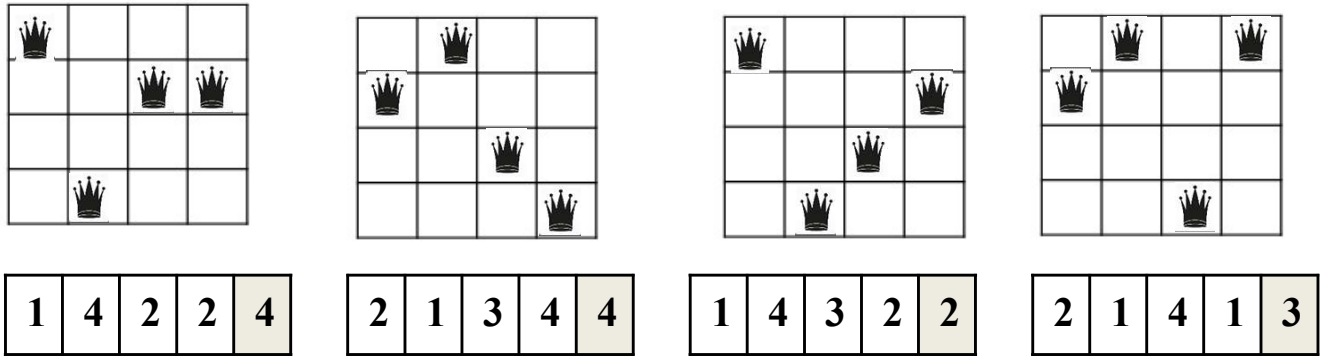
Step 02: Generate initial population

1. Select ‘k’ random states – **Initialization : k=4**
2. Evaluate the fitness value all states
3. If anyone of the state’s has achieved the threshold fitness value or threshold new states or no change is seen than previous iteration then the algorithm stops
4. Else, use roulette wheel mechanism to select pair/s
5. Pairs selected produces new state (successor) by crossover
6. Successor is allowed to mutate
7. Repeat from Step 2



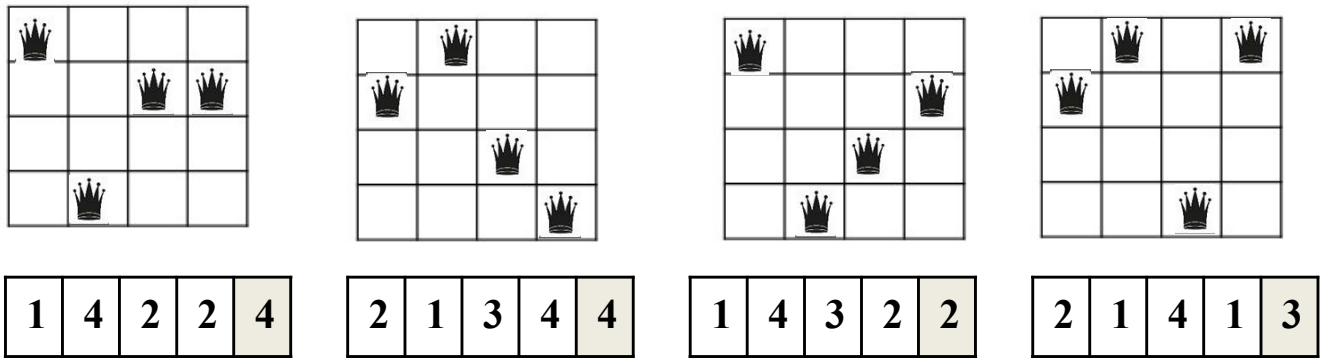
Step 03: Apply Fitness Function

1. Select ‘k’ random states – **Initialization : k=4**
2. Evaluate the fitness value all states : Maximizing function : No of Non-attacking pairs Queens → Threshold = 6
3. If anyone of the state’s has achieved the threshold fitness value or threshold new states or no change is seen than previous iteration then the algorithm stops
4. Else, use roulette wheel mechanism to select pair/s
5. Pairs selected produces new state (successor) by crossover
6. Successor is allowed to mutate
7. Repeat from Step 2



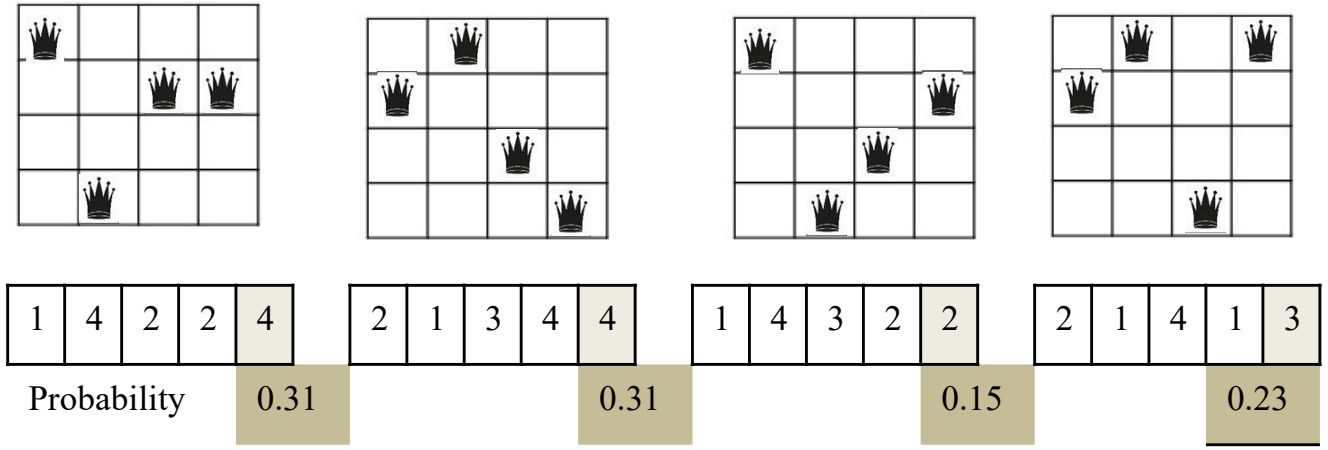
Step 03: Apply Fitness Function

1. Select ‘k’ random states – **Initialization : k=4**
2. Evaluate the fitness value all states : Maximizing function : No of Non-attacking pairs Queens → Threshold = 6
3. If anyone of the state’s has achieved the threshold fitness value or threshold new states or no change is seen than previous iteration then the algorithm stops
4. Else, use roulette wheel mechanism to select pair/s
5. Pairs selected produces new state (successor) by crossover
6. Successor is allowed to mutate
7. Repeat from Step 2



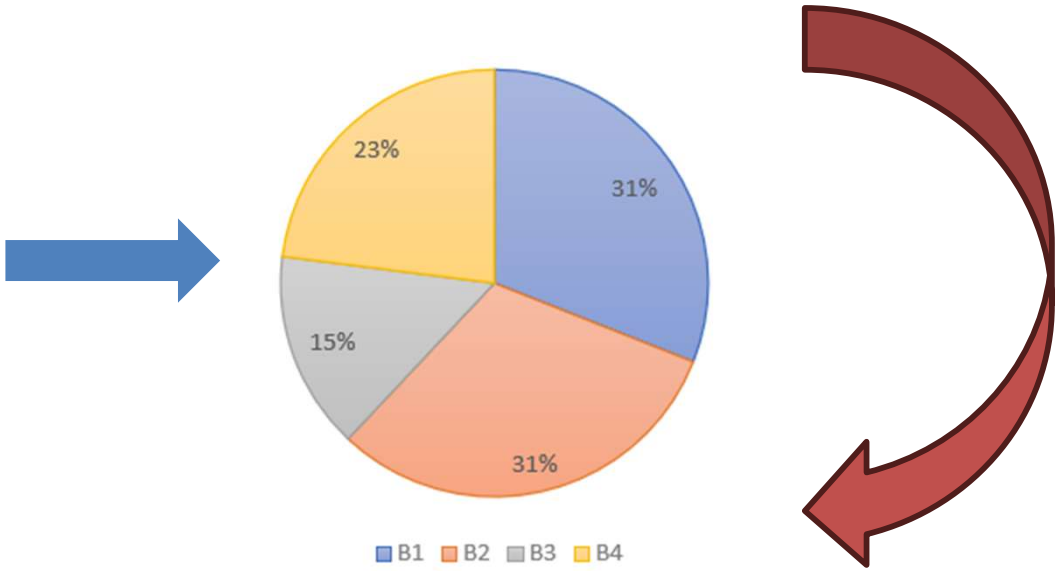
Step 04: Selection

1. Select ‘k’ random states – **Initialization : k=4**
2. Evaluate the fitness value all states : Maximizing function : No of Non-attacking pairs Queens → Threshold = 6
3. If anyone of the state’s has achieved the threshold fitness value or threshold new states or no change is seen than previous iteration then the algorithm stops
4. Else, use roulette wheel mechanism to select pair/s
5. Pairs selected produces new state (successor) by crossover
6. Successor is allowed to mutate
7. Repeat from Step 2



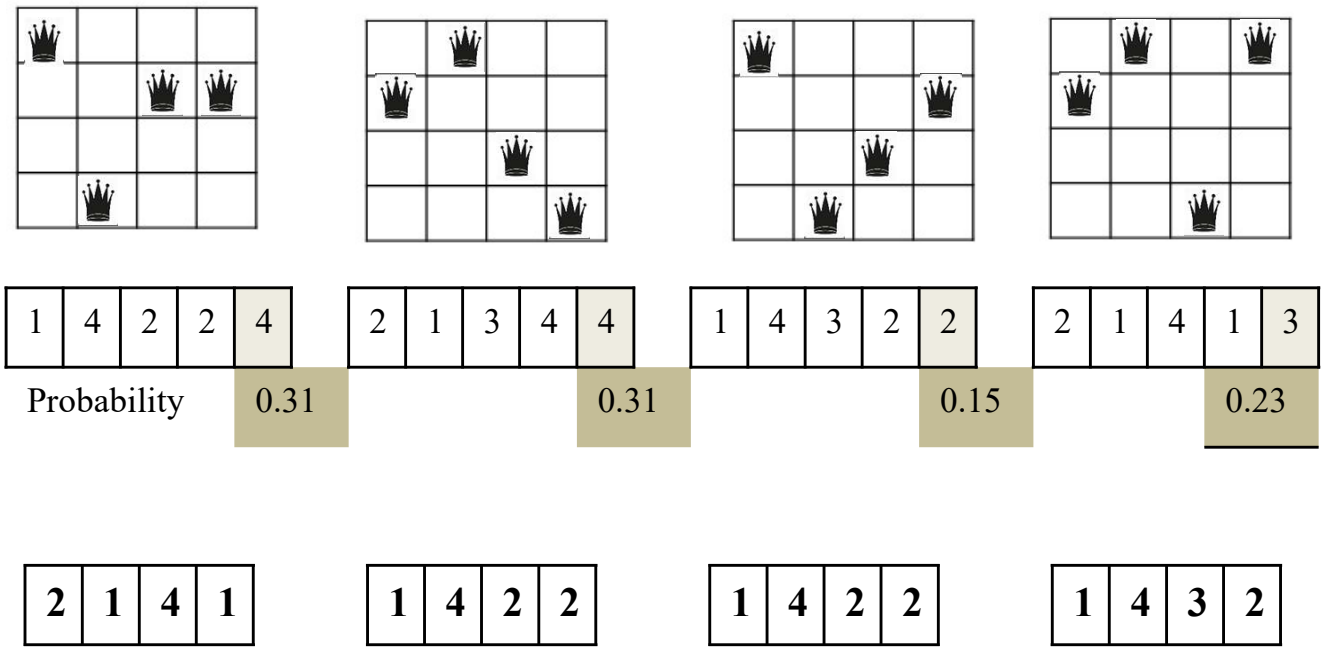
Step 04: Selection

- There are various methods of selection.
- Roulette wheel, tournament, rank, etc.
- Population is divided on a wheel according to their respective percentages of fitness and two fixed points are placed.
- Wheel is spun and those individual are selected at which the fixed points are pointing when the wheel stops.



Step 04: Selection

Sample winners of game:1,2,3,4 : B4, B1, B1, B3
So, <B4,B1> and <B1, B3> are the selected parents.



Crossover Strategies

There are several possible crossover Strategies:

- Randomly select a single point for a crossover
- Multi point crossover
- Uniform crossover

Single point crossover

Two point crossover (Multi point crossover)

Crossover Strategies

There are several possible crossover Strategies:

- Randomly select a single point for a crossover
- Multi point crossover
- Uniform crossover

Uniform crossover

- A random subset is chosen
- The subset is taken from parent 1 and the other bits from parent 2.

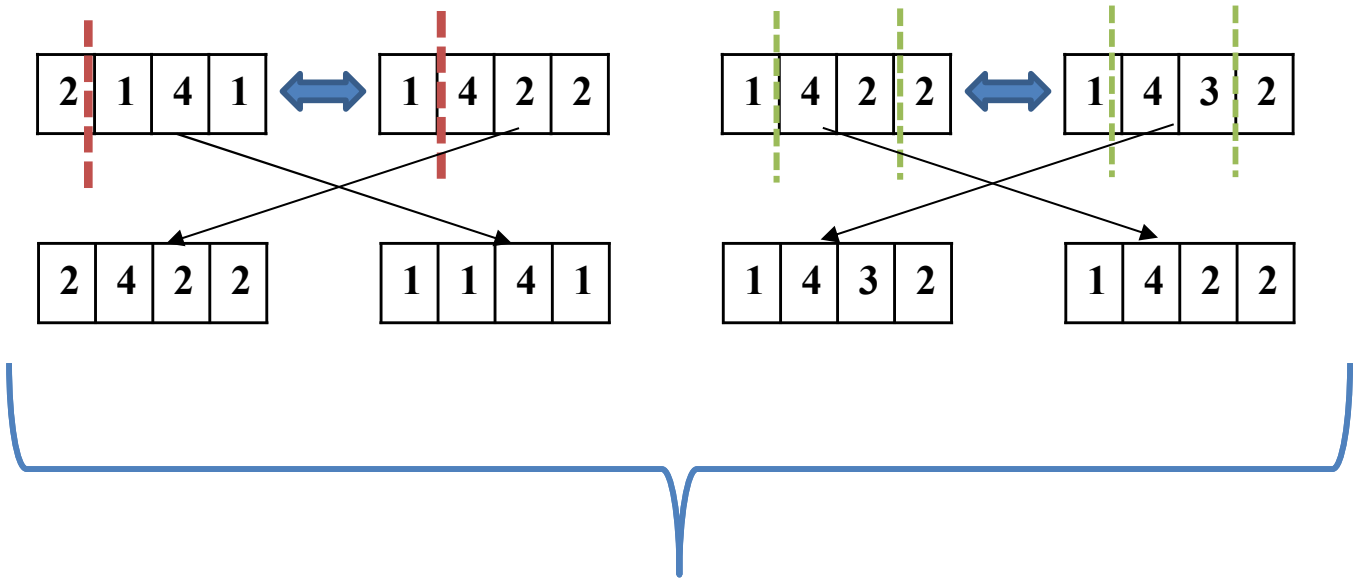
Subset: **BAABBAABBB** **(Randomly generated)**

Parents: 1010001110 0011010010

Offspring: 0011001010 1010010110

Step 05: Crossover

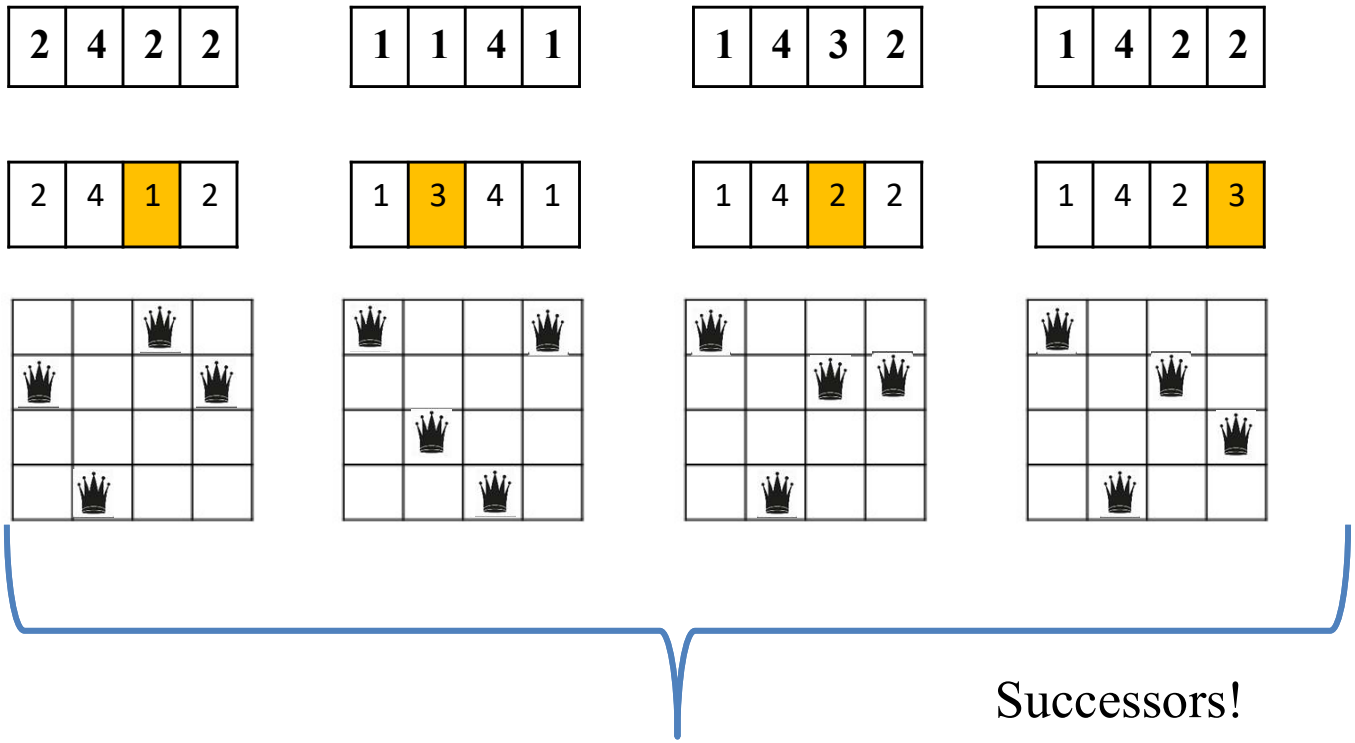
1. Select ‘k’ random states – **Initialization : k=4**
2. Evaluate the fitness value all states : Maximizing function : No of Non-attacking pairs Queens → Threshold = 6
3. If anyone of the state’s has achieved the threshold fitness value or threshold new states or no change is seen than previous iteration then the algorithm stops
4. Else, use roulette wheel mechanism to select pair/s
5. Pairs selected produces new state (successor) by crossover
6. Successor is allowed to mutate
7. Repeat from Step 2



Successors!

Step 06: Mutation

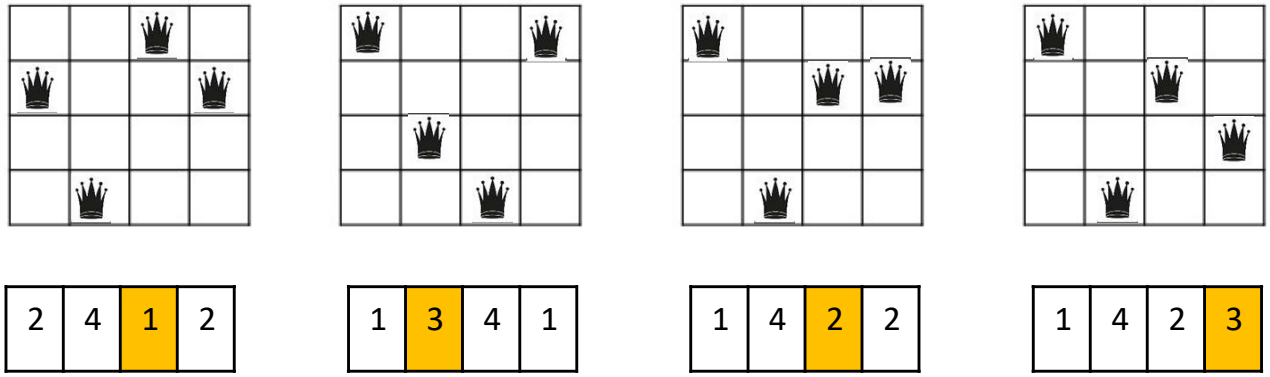
1. Select ‘k’ random states – **Initialization : k=4**
2. Evaluate the fitness value all states : Maximizing function : No of Non-attacking pairs Queens → Threshold = 6
3. If anyone of the state’s has achieved the threshold fitness value or threshold new states or no change is seen than previous iteration then the algorithm stops
4. Else, use roulette wheel mechanism to select pair/s
5. Pairs selected produces new state (successor) by crossover
6. Successor is allowed to mutate (Change Gene)
7. Repeat from Step 2



Iteration 1 Completed

Step 07: Repeat!

1. Select ‘k’ random states – **Initialization : k=4**
2. Evaluate the fitness value all states : Maximizing function : No of Non-attacking pairs Queens → Threshold = 6
3. If anyone of the state’s has achieved the threshold fitness value or threshold new states or no change is seen than previous iteration then the algorithm stops
4. Else, use roulette wheel mechanism to select pair/s
5. Pairs selected produces new state (successor) by crossover
6. Successor is allowed to mutate (Change Gene)
7. Repeat from Step 2



- Calculate their fitness, if anyone has fitness as 6, STOP. Else
- Calculate probabilities, apply roulette wheel, perform cross over and mutation!

Genetic Algorithm

function GENETIC-ALGORITHM(*population*, FITNESS-FN) **returns** an individual
  inputs: *population*, a set of individuals
    FITNESS-FN, a function that measures the fitness of an individual

  repeat
    new_population ← empty set
    for $i = 1$ **to** SIZE(*population*) **do**
      x ← RANDOM-SELECTION(*population*, FITNESS-FN)
      y ← RANDOM-SELECTION(*population*, FITNESS-FN)
      child ← REPRODUCE(x, y)
      if (small random probability) **then** *child* ← MUTATE(*child*)
      add *child* to *new_population*
    population ← *new_population*
  until some individual is fit enough, or enough time has elapsed
  return the best individual in *population*, according to FITNESS-FN

---

function REPRODUCE(x, y) **returns** an individual
  inputs: x, y, parent individuals

  n ← LENGTH(x); c ← random number from 1 to n
  return APPEND(SUBSTRING($x, 1, c$), SUBSTRING($y, c + 1, n$))

Genetic Algorithm

Techniques:

1. Design of the fitness function
2. Diversity in the population to be accounted
3. Randomization

Application:

- Creative tasks
- Exploratory in nature
- Planning problem
- Static Applications

Online vs Offline Search

- **So far, most search algorithms assume an offline setting**
  - In offline search, the agent has a complete model of the problem before acting.
- **Offline search means:**
  - The full state space is known or can be generated in advance.
  - The agent can simulate possible actions before taking them.
  - A complete plan is produced first, then executed.
  - Examples: Breadth-First Search, Uniform Cost Search, A*, Hill Climbing, Local Beam Search.
- **Example**
  - A route-planning system knows the full map, road connections, and costs before suggesting a path.

Online Search Agents

- **Online search agents do not know the full environment in advance.**
  - They must act, observe the result, update their knowledge, and continue searching.
- **Online search means:**
  - The agent discovers the environment while moving through it.
  - It may not know all possible states or action outcomes initially.
  - It must balance exploration and progress toward the goal.
  - Planning and execution happen together.
  - Useful when the environment is unknown, dynamic, or too large to fully model.
- **Example**
  - A robot exploring an unfamiliar building cannot plan the full route in advance. It moves, senses nearby paths, updates its map, and decides the next action.

Ant Colony Optimization

General pseudo-code

Procedure ***ACO***

Schedule Activities

Initialization

Construction

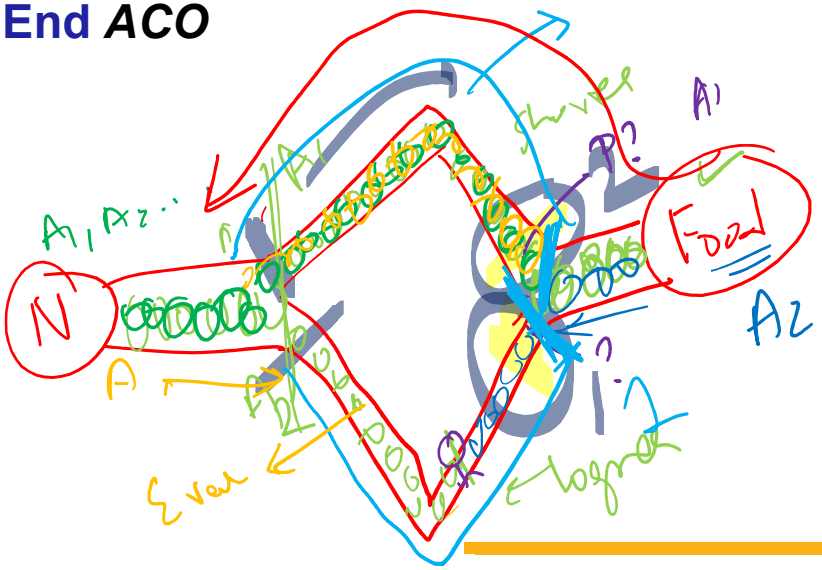
Update Pheromone

Daemon Actions {optional}

// local search, elitism

End schedule activities

End ***ACO***



Parameters used in ACO

Parameter	Description
N	Total No of ants ; N>1
τ_o	Initial pheromone amount
τ_{ij}	Amount of pheromone deposited while traversing from i to j
η_{ij}	Cost of link (i,j)
α	Importance coefficient of pheromone intensity
β	Importance coefficient of route cost
ρ	evaporation co-efficient; 0<ρ<1
$visit_k$	Visited nodes table of k^{th} ant
Q	Importance - Constant value pertaining to pheromone trail
f_k	Route cost obtained by ant k

ACO Steps

Initialization

- Place predefined number of ants on starting point
- Set values for parameters α, β, ρ.
- Set τ_0 to 0.

– Construction

- Compute the next node transition probability

$$NTP_{ij} = \frac{(\tau_{ij})^{\alpha} (\eta_{ij})^{\beta}}{\sum_{h \notin visit_k} (\tau_{ih})^{\alpha} (\eta_{ih})^{\beta}}$$

ACO Steps

Pheromone updation:

- Pheromone reinforcement & pheromone evaporation
- Direct impact on the exploitation (enhancing found food path) & exploration (discovering new path) of ant algorithms

$$\tau_{ij}^{new} = (1-\rho)\tau_{ij}^{old} + \Delta\tau_{ij}^{k}$$

- Amount of pheromone deposited on (i,j) by kth at that timestamp is given by

$$\Delta\tau_{ij}^{k} = \begin{cases} \dfrac{Q}{\boldsymbol{f_k}} & if\ k^{th} ant\ passes\ i\ and\ j \\ 0 & otherwise \end{cases}$$

- **Stopping criteria:** reaching predetermined number of iterations

Problem: Reaching pre-determined number of iterations before reaching destination leading to ant drop

We will inch like an ant towards our goal in the next class …

Exercise 1

- Suppose local beam search has k = 3 and the following current states:

Current State	Heuristic Value
S1	12
S2	15
S3	10

- Their successors are:

Successor	Heuristic Value
A	16
B	11
C	14
D	18
E	13
F	17

If the goal is to maximize, which three states will be selected for the next beam?

Exercise 2

1. Given two parent chromosomes:

- Parent 1: [1, 2, 3, 4, 5, 6, 7, 8]
  Parent 2: [8, 7, 6, 5, 4, 3, 2, 1]
- Using single-point crossover after the 4th position, generate two children.

2. Suppose the fitness values of four individuals are:

Individual	Fitness
A	10
B	30
C	40
D	20

Which individual has the highest chance of being selected? Why?

Exercise 2 Continued

Part B: Relaxed Problems

A relaxed problem is obtained by removing one or more constraints from the original problem.

Consider the following relaxed versions of the warehouse problem:

- **Relaxation 1:** Ignore all locked-door constraints. The robot can pass through any corridor.
- **Relaxation 2:** Ignore package collection order. Estimate the cost only as the shortest distance from the current room to the exit.
- **Relaxation 3:** Assume the robot can collect any remaining package instantly once it enters the same connected component, but it must still move to the exit.
- **Relaxation 4:** Ignore the robot's current location and estimate only the minimum cost needed to connect all uncollected package locations and the exit.

Answer the following:

1. For each relaxed problem, explain which constraint has been removed.
2. Which relaxed problem is likely to give the weakest heuristic?
3. Which relaxed problem is likely to give the strongest heuristic?
4. Why does the solution cost of a relaxed problem give an admissible heuristic for the original problem?
5. Construct few heuristics by picking some relaxed version of the problem.

ACI: Disclaimer and Acknowledgements

- Few content for these slides may have been obtained from prescribed books and various other source(s) on the Internet.
- We hereby acknowledge all the contributors for their material and inputs and gratefully acknowledge people others who made their course materials freely available online. We have provided source information wherever necessary.
- We have added and modified the content to suit the requirements of the class dynamics & live session's lecture delivery flow for presentation.
- This *is not a full-fledged* reading materials. Students are requested to refer to the textbook w.r.t detailed content as per the course handout as shared over e-learning portal - Taxila.
- **Slide Source / Preparation / Review:**
- From BITS Pilani WILP**:** Prof. Rajavadhana, Prof. Indumathi, Prof. Sangeetha and Prof Parthasarathy PD
- From BITS On-campus & External : Mr.Santosh GSK

Thank You for your time & attention !